

ออกสอบ V3



ก่อนสอบอ่านชีตนี้ทวนสักรอบก่อนได้อ

สมทบค่ากาแฟ ได้ที่นี่



Nattawut Rattanabucha



Hashing

* ออกสอบ!

hash function ในอุดมคติ ต้อง มีดังนี้ :

1. ง่ายต่อการประมวลผล
2. ต้องมั่นใจว่า ข้อมูลที่แตกต่างกัน 2 ตัว เมื่อผ่านฟังก์ชันแล้วได้ ตำแหน่งใน Array คนละตำแหน่ง เป็นไปไม่ได้ในโลกความเป็นจริงเพราะ
 1. ช่องใน Hash Table มีอยู่อย่างจำกัด
 2. จำนวนช่องใน Hash Table ใช้แล้วหมดไป
3. ข้อมูลแต่ละตัวต้องกระจายได้ทั่วทั้ง

* ออกสอบ

Wrap around ใน hashing คือ เมื่อคำนวณถึงตำแหน่งท้ายของ array แล้วให้วนกลับมาตำแหน่งที่ 0 ใหม่ ก็คือ การ mod TableSize อีกรอบนี้

* อาจออกสอบ

ทำงานใน for loop เท่ากับ 3 รอบ เพราะ รอบที่ 4 เြื่อนใจเป็นเท็จ ออกจาก for loop ตัวแปร i มีค่าเท่ากับ 221 ออกจาก for loop ตัวแปร hashVal มีค่าเท่ากับ 221

* อาจออกสอบ

Perfect Square (กำลังสองสมบูรณ์)

$0^2, 1^2, 2^2, 3^2, 4^2, 5^2, 6^2, 7^2, 8^2, 9^2$
 $\downarrow \downarrow \downarrow \downarrow \downarrow \downarrow \downarrow \downarrow \downarrow \downarrow$ จะได้เป็น
 $0, 1, 4, 9, 16, 25, 36, 49, 64, 81$
 ซ้ำกันครั้งที่ 1 36 กับ 16
 ซ้ำกันครั้งที่ 2 49 กับ 9

Priority Queue หรือ Heap

* ออกสอบ

Complete Binary Tree มีความสูง = 15
 จำนวนโหนดอย่างน้อย $2^{15} = 32768$ โหนด
 จำนวนโหนดอย่างมาก $2^{16} - 1 = 65535$ โหนด
 ถ้าโจทย์ไม่มี Complete ให้สุ่มนี้ไม่ได้

* ออกสอบ

อธิบายหลักการ exchangeable?
ตอบ สามารถเปลี่ยนชื่อของฟังก์ชันได้เลยเพราะไม่กระทบส่วนอื่นของ main program

การทำงานของฟังก์ชัน Insert

* ออกสอบ

hole = 11
 for loop รอบที่ 1 Hole = 11 > 1 จริง เปรียบเทียบ 14 < 31 จริง
 for loop รอบที่ 2 Hole = 5 > 1 จริง เปรียบเทียบ 14 < 21 จริง
 for loop รอบที่ 3 Hole = 2 > 1 จริง เปรียบเทียบ 14 < 13 เท็จ
 ออกจาก for loop ตัวแปร hole มีค่าเท่ากับ 2
 for loop ทำงาน 3 รอบ ประโยคคำสั่งที่อยู่ใน for loop ทำงาน 2 รอบ

การทำงานของฟังก์ชัน deleteMin

* ออกสอบ

hole = 11 tmp 31
 for loop รอบที่ 1 Hole = 1, child = 2 เปรียบเทียบ 16 < 14 เท็จ
 for loop รอบที่ 2 Hole = 2, child = 4 เปรียบเทียบ 21 < 19 เท็จ ไม่ทำ child++;
 for loop รอบที่ 3 Hole = 4, child = 8 เปรียบเทียบ 26 < 65 จริง ทำ child++;
 for loop รอบที่ 4 Hole = 9, child = 9
 for loop ทำงาน 3 รอบ

Sorting

* ออกสอบ

อธิบายหลักการ exchangeable?
ตอบ สามารถเปลี่ยนชื่อของฟังก์ชันได้เลยเพราะไม่กระทบส่วนอื่นของ main program

Graphs

* ออกสอบ

Complete Graph ที่มี 10 vertex (จุด)
edge สี่เหลี่ยมน้อยก็เส้น

* สูตร $n \times (n-1)/2$

ตอบ $10 \times (9)/2 = 45$ เส้น#

* ออกสอบ

กำหนด Direct Graph มี 10 Vertex มี edge 20 เส้น ถามว่า
adjacency matrix ใช้กี่ช่อง

ตอบ $\Theta(|V|^2) = 10^2 = 100$ ช่อง

* ออกสอบ

กำหนด Direct Graph มี 10 Vertex มี edge 20 เส้น ถามว่า
adjacency list ใช้กี่ช่อง

ตอบ $O(|E| + |V|) = 20 + 10 = 30$ เส้น

แนวข้อสอบ

75 คะแนนเต็ม

ข้อที่ 1. Hashing (15 คะแนน)
- TableSize มากกว่า 12

2. Heap

- คล้าย Assignment 3
- มีข้อ Insert และ deleteMin เล็กน้อย
- มีเงื่อนไขของการทำงาน
- Hole มีค่าเท่าไร
- Child มีค่าเท่าไร
- เปรียบเทียบ เลขโดด กับเลขโดด

3. InsertionSort

- จงเขียนผลลัพธ์ After P = ?
- ค่าของตัวแปร tmp
- ค่าทั้งหมดของ Position Move

4. Shellsort

- จะให้เลข increment sequence เช่น 1,3,5 หรือ 2,4,6
- จงเขียนผลการการทำงานหลังจาก After Sort 1 หรือ 2

5. Topological Sort

- จงเขียนค่า Indegree Array

6. Shortest Path

- จงเขียนค่าในตารางออกมา

7. คำถามเชิงอธิบาย

- จงเขียนประโยคคำสั่งเพื่อใช้แทน Function _____
- คำนวณ edge ใน Complete Graph
- เขียนขั้นตอนวิธี

โน้ตอู้ง V3



ก่อนสอบอ่านชีตนี้ทวนสั้กรอบก่อนเด้อ

สมทบค่ากาแฟ ได้ที่นี่



Nattawut Rattanabucha



Hashing คือการนำข้อมูลเข้าไปใน Array

hash Table : อาร์เรย์ที่เก็บข้อมูลที่มีจำนวนช่องแน่นอน
จำนวนช่องคือ TableSize

การค้นหาคำข้อมูลจะหาโดยชื่อ : Key โดย Key จะถูก Map
เพื่อเอาเลขที่เป็นตำแหน่งของอาร์เรย์โดยจะเริ่มตั้งแต่ตำแหน่ง
ที่ 0 ถึง TableSize-1 คือ Index array

หัวข้อที่จะพิจารณา ดังนี้

1. วิธีการเลือก Hash function
2. ตัดสินใจว่าจะทำอย่างไรข้อมูลที่ผ่านมา Hash function
ได้ค่าเหมือนกัน
3. การพิจารณา TableSize

Hash function ทางทฤษฎี
แบ่งออกมาได้ 2 ประเด็น

1. ถ้าข้อมูลเป็นเลขจำนวนเต็ม (Integer)

1.1 hash function จะ return Key % TableSize

1.2 TableSize ควรเป็นเลขจำนวนเฉพาะ

2 3 5 7 11 13 17 19 23
29 31 37 41 43 47 53 59 61
67 71 73 79 83 89 97

2. ถ้าข้อมูลเป็นสตริง (String)

2.1 นำค่าของ ASCII ของตัวอักษร มาบวกเข้าด้วยกัน
ได้ผลลัพธ์ และ Mod ด้วย TableSize

Division = การหารธรรมดา "/"
Modulo = การหารเอาเศษ "%"

ยกตัวอย่าง 23 / 7 $23 = \text{ตัวตั้ง}$
7 = ตัวหาร
$$\begin{array}{r} 3 \\ 7 \overline{) 23} \\ \underline{21} \\ 2 \end{array}$$
$$23 / 7 = 3$$
$$23 \% 7 = 2$$

* ออกข้อสอบ!

hash function ในอุดมคติ ต้องมีดังนี้ :

1. ง่ายต่อการประมวลผล
2. ตัดสินใจว่า ข้อมูลที่แตกต่างกัน 2 ตัว
เมื่อผ่านฟังก์ชันแล้วได้ ตำแหน่งใน Array
คนละตำแหน่ง เป็นไปไม่ได้ในโลกความจริงเพราะ
1. ช่องใน Hash Table มีอยู่อย่างจำกัด
2. จำนวนช่องใน Hash Table ใช้แล้วหมดไป
3. ข้อมูลแต่ละตัวต้องกระจายได้ทั่วทั้ง

ASCII TABLE

Decimal	Hexadecimal	Binary	Octal	Char	Decimal	Hexadecimal	Binary	Octal	Char	Decimal	Hexadecimal	Binary	Octal	Char
0	0	0	0	[NULL]	48	30	110000	60	0	96	60	1100000	140	
1	1	1	1	[START OF HEADING]	49	31	110001	61	1	97	61	1100001	141	a
2	2	10	2	[START OF TEXT]	50	32	110010	62	2	98	62	1100010	142	b
3	3	11	3	[END OF TEXT]	51	33	110011	63	3	99	63	1100011	143	c
4	4	100	4	[END OF TRANSMISSION]	52	34	110100	64	4	100	64	1100100	144	d
5	5	101	5	[ENQUIRY]	53	35	110101	65	5	101	65	1100101	145	e
6	6	110	6	[ACKNOWLEDGE]	54	36	110110	66	6	102	66	1100110	146	f
7	7	111	7	[BELL]	55	37	110111	67	7	103	67	1100111	147	g
8	8	1000	10	[BACKSPACE]	56	38	111000	70	8	104	68	1101000	150	h
9	9	1001	11	[HORIZONTAL TAB]	57	39	111001	71	9	105	69	1101001	151	i
10	A	1010	12	[LINE FEED]	58	3A	111010	72	:	106	6A	1101010	152	j
11	B	1011	13	[VERTICAL TAB]	59	3B	111011	73	.	107	6B	1101011	153	k
12	C	1100	14	[FORM FEED]	60	3C	111100	74	<	108	6C	1101100	154	l
13	D	1101	15	[CARRIAGE RETURN]	61	3D	111101	75	=	109	6D	1101101	155	m
14	E	1110	16	[SHIFT OUT]	62	3E	111110	76	>	110	6E	1101110	156	n
15	F	1111	17	[SHIFT IN]	63	3F	111111	77	?	111	6F	1101111	157	o
16	10	10000	20	[DATA LINK ESCAPE]	64	40	1000000	100	@	112	70	1110000	160	p
17	11	10001	21	[DEVICE CONTROL 1]	65	41	1000001	101	A	113	71	1110001	161	q
18	12	10010	22	[DEVICE CONTROL 2]	66	42	1000010	102	B	114	72	1110010	162	r
19	13	10011	23	[DEVICE CONTROL 3]	67	43	1000011	103	C	115	73	1110011	163	s
20	14	10100	24	[DEVICE CONTROL 4]	68	44	1000100	104	D	116	74	1110100	164	t
21	15	10101	25	[NEGATIVE ACKNOWLEDGE]	69	45	1000101	105	E	117	75	1110101	165	u
22	16	10110	26	[SYNCHRONOUS IDLE]	70	46	1000110	106	F	118	76	1110110	166	v
23	17	10111	27	[END OF TRANS. BLOCK]	71	47	1000111	107	G	119	77	1110111	167	w
24	18	11000	30	[CANCEL]	72	48	1001000	110	H	120	78	1111000	170	x
25	19	11001	31	[END OF MEDIUM]	73	49	1001001	111	I	121	79	1111001	171	y
26	1A	11010	32	[SUBSTITUTE]	74	4A	1001010	112	J	122	7A	1111010	172	z
27	1B	11011	33	[ESCAPE]	75	4B	1001011	113	K	123	7B	1111011	173	{
28	1C	11100	34	[FILE SEPARATOR]	76	4C	1001100	114	L	124	7C	1111100	174	
29	1D	11101	35	[GROUP SEPARATOR]	77	4D	1001101	115	M	125	7D	1111101	175	}
30	1E	11110	36	[RECORD SEPARATOR]	78	4E	1001110	116	N	126	7E	1111110	176	~
31	1F	11111	37	[UNIT SEPARATOR]	79	4F	1001111	117	O	127	7F	1111111	177	[DEL]
32	20	100000	40	[SPACE]	80	50	1010000	120	P					
33	21	100001	41	!	81	51	1010001	121	Q					
34	22	100010	42	"	82	52	1010010	122	R					
35	23	100011	43	#	83	53	1010011	123	S					
36	24	100100	44	\$	84	54	1010100	124	T					
37	25	100101	45	%	85	55	1010101	125	U					
38	26	100110	46	&	86	56	1010110	126	V					
39	27	100111	47	'	87	57	1010111	127	W					
40	28	101000	50	(	88	58	1011000	130	X					
41	29	101001	51	)	89	59	1011001	131	Y					
42	2A	101010	52	*	90	5A	1011010	132	Z					
43	2B	101011	53	+	91	5B	1011011	133	[					
44	2C	101100	54	,	92	5C	1011100	134	\					
45	2D	101101	55	-	93	5D	1011101	135	]					
46	2E	101110	56	.	94	5E	1011110	136	^					
47	2F	101111	57	/	95	5F	1011111	137	_					

Main Program

```
1 int hash(const string& key, int tableSize);
2 int main() {
3     string a[7];
4     int x;
5     x = hash("ANN", 7);
6     a[x] = ("ANN");
7     return 0; }
8 int hash(const string & key, int tableSize) {
9     int hashVal = 0;
10    for (int i=0; i<key.length(); i++)
11        hashVal += key[i];
12    return hashVal % tableSize; }
```

บรรทัดที่ 5

```
int hash( const string &key, int tableSize )
{
    int hashVal = 0;
    for( int i = 0; i < key.length( ); i++ )
        hashVal += key[i];

    return hashVal % tableSize;
}
```



A หรือ N ได้เลขมาจากไหน?
เลขมาจากฐานสิบใน ASCII
เช่น 'A' = 65 , 'N' = 78

ผลลัพธ์

hashVal = 0
for loop รอบแรก i = 0, 0 < 3

hashVal = hashVal + key[0];
= 0 + 'A';
= 0 + 65; ฐานสิบ

hashVal = 65
for loop รอบสอง i = 1, 1 < 3

hashVal = hashVal + key[1];
= 65 + "N";
= 65 + 78; ฐานสิบ

hashVal = 221
for loop รอบที่ i = 3, 3 < 3

ออกจาก for loop ไปทำบรรทัด
return 221 % 7 = 4;

hashVal = 143
for loop รอบสาม i = 2, 2 < 3

hashVal = hashVal + key[2];
= 143 + "N";
= 143 + 78; ฐานสิบ

[0]	
1	
2	
3	
4	"ANN"
5	
6	

* อาจออกสอบ

ทำงานใน for loop เท่ากับ 3 รอบ เพราะ รอบที่ 4 เြือนใจไปเพิก
ออกจาก for loop ตัวแปร i มีค่าเท่ากับ 3 ไม่มี
ออกจาก for loop ตัวแปร hashVal มีค่าเท่ากับ 221

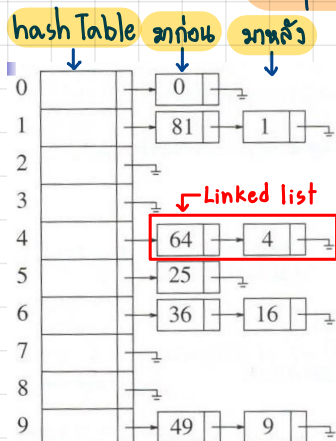
ถ้า TableSize มีขนาดใหญ่พอสมควร
10000 พลังงาน hash จะไม่ช่วยกระจาย
ข้อมูลได้

Collision การชนกัน

คือ ข้อมูลที่นำเข้ามาผ่าน Hash function เดียวกัน และได้ตำแหน่ง Array เดียวกัน เรียกว่าการชนกัน
มีวิธีแก้ปัญหานี้ 2 วิธี

1. Separate chaining (แยกออกจาก hash Table)
2. Open Addressing

Separate chaining



คือ นำค่าที่ Mod ได้มาใส่ Linked list
และ นำ Address ของ Linked list ไป
ใส่ใน Hash Table ตำแหน่งที่ Mod ได้
(แต่ไม่จำเป็นต้องใช้เพราะยุ่งยาก ในการ
เก็บข้อมูลถึง 2 ชนิด)

* อาจออกสอบ

Perfect Square (กำลังสองสมบูรณ์)

$0^2, 1^2, 2^2, 3^2, 4^2, 5^2, 6^2, 7^2, 8^2, 9^2$
↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓ จะได้เป็น
0, 1, 4, 9, 16, 25, 36, 49, 64, 81
ชนกันครั้งที่ 1 36 กับ 16
ชนกันครั้งที่ 2 49 กับ 9

Open Addressing คือ ถ้ามีที่ชนกันเกิดขึ้น จะหาช่องว่างใหม่แทน

มีวิธี แตกออกเป็น 3 วิธี

สูตรคือ

$$h_i(x) = (\text{hash}(x) + F(i)) \bmod \text{TableSize}$$

$$F(0) = 0$$

1. Linear Probing (สมการเชิงเส้น ไม่มีเลขยกกำลัง)

ตัวอย่าง Insert keys {89, 18, 49, 58, 69}

89 $h_i(89) = (\text{hash}(x) + F(i)) \bmod \text{TableSize}$; $F(0) = 0$
 $h_0(89) = ((89 \% 10) + F(0)) \% 10$; $F(i) = i$
 $= (9 + 0) = 9 \% 10 = 9$; ← ตำแหน่งในอาร์เรย์

18 $h_i(18) = (\text{hash}(x) + F(i)) \bmod \text{TableSize}$; $F(0) = 0$
 $h_0(18) = ((18 \% 10) + F(0)) \% 10$; $F(i) = i$
 $= (8 + 0) = 8 \% 10 = 8$; ← ตำแหน่งในอาร์เรย์

49 $h_i(x) = (\text{hash}(x) + F(i)) \bmod \text{TableSize}$; $F(0) = 0$
 $h_0(x) = ((49 \% 10) + F(0)) \% 10$; $F(i) = i$
 $= (9 + 0) = 9 \% 10 = 9$; ← ตำแหน่งในอาร์เรย์
 มีที่ชนกันที่ตำแหน่ง 9 ให้ $i += 1$

49 $h_i(x) = (\text{hash}(x) + F(i)) \bmod \text{TableSize}$; $F(0) = 0$
 $h_0(49) = ((49 \% 10) + F(1)) \% 10$; $F(1) = 1$
 $= (9 + 1) = 10 \% 10 = 0$; ← ตำแหน่งในอาร์เรย์

89, 18, 49 ชนกันและแก้ที่ชนกัน 1 ครั้ง
 ถ้าใส่ 89, 18, 49, 58, 69 ชนกันและแก้ที่ชนกัน 7 ครั้ง

	89	18	49		
Empty Table	After 89	After 18	After 49	After 58	After 69
0			49	49	49
1				58	58
2					69
3					
4					
5					
6					
7					
8		18	18	18	18
9	89	89	89	89	89

ปัญหาของ Linear Probing

- ใช้เวลานาน
- Primary Clustering (เกาะกลุ่ม)

*** ออกสอบ**

Wrap around ใน hashing คือ เมื่อคำนวณถึงตำแหน่งท้ายของ Array แล้วให้วนกลับมาตำแหน่งที่ 0 ใหม่ ก็คือ mod TableSize อีกรอบนี้

2. Quadratic Probing (สมการเลขยกกำลัง)

สูตร คือ

Ex : $F(i) = i^2$

จะช่วยให้เรื่องเกาะกลุ่มของข้อมูล
 แต่จะเกิดปัญหา Secondary clustering (ที่ชนกันขั้นที่ 2) มีหลายกลุ่ม

ยกตัวอย่าง : 89, 18, 49, 58, 69

$h_i(x) = (\text{hash}(x) + F(i)) \bmod \text{TableSize}$; $F(0) = 0$
 $h_0(89) = ((89 \% 10) + F(0)) \% 10$; $F(0) = 0$
 $h_0(89) = (9 + 0) = 9 \% 10 = 9$

$h_0(18) = ((18 \% 10) + F(0)) \% 10$; $F(0) = 0$
 $h_0(18) = (8 + 0) = 8 \% 10 = 8$; $F(0) = 0$

$h_0(49) = ((49 \% 10) + F(0)) \% 10$; $F(0) = 0$
 $h_0(49) = (9 + 0) = 9 \% 10 = 9$; $F(i) = i^2$
 $h_1(49) = (9 + 1) = 10 \% 10 = 0$; $F(1) = 1^2 = 1$

49 ชนกันและแก้ที่ชนกัน 1 ครั้ง

	89	18	49	58	69
Empty Table	After 89	After 18	After 49	After 58	After 69
0			49	49	49
1					
2				58	58
3					69
4					
5					
6					
7					
8		18	18	18	18
9	89	89	89	89	89

$$h_0(56) = ((56 \div 10) + F(0)) \% 10;$$

$$h_0(56) = (8 + 0) = 8 \div 10 = 8$$

$$h_1(56) = (8 + 1) = 9 \div 10 = 9$$

$$h_1(56) = (8 + 4) = 12 \div 10 = 2$$

56 ชนกันและแก้การชนกัน 2 ครั้ง

$$F(0) = 0$$

$$F(i) = i^2$$

$$F(1) = 1^2 = 1$$

$$F(2) = 2^2 = 4$$

	Empty Table	After 89	After 18	After 49	After 58	After 69
0				49	49	49
1						
2					58	58
3						69
4						
5						
6						
7						
8			18	18	18	18
9		89	89	89	89	89

$$h_0(69) = ((69 \div 10) + F(0)) \% 10;$$

$$h_0(69) = (9 + 0) = 9 \div 10 = 9$$

$$h_1(69) = (9 + 1) = 10 \div 10 = 0$$

$$h_1(69) = (9 + 4) = 13 \div 10 = 3$$

69 ชนกันและแก้การชนกัน 2 ครั้ง

$$F(0) = 0$$

$$F(i) = i^2$$

$$F(1) = 1^2 = 1$$

$$F(2) = 2^2 = 4$$

3. Double Hashing พังจัน Hash ที่สอง

R คือ เลขจำนวนเฉพาะที่มากที่สุด แต่ต้องน้อยกว่า TableSize

เช่น TableSize = 10 ค่า R คือ 7

สูตร คือ

$$F(i) = i * \text{hash}_2(x)$$

$$\text{hash}_2(x) = R - (X \bmod R)$$

ตัวอย่าง ใส่ 60 ลงใน Hash Table :

ขั้น 0 $h_0(60) = (\text{hash}(x) + f(i)) \% \text{TableSize}; f(0) = 0$

$$h_0(60) = (60 \div 10) + 0) \div 10;$$

$$h_0(60) = (0 + 0) \div 10 = 0; \leftarrow \text{ชนให้ } i+1$$

$$F(1) = 1 * \text{hash}_2(60)$$

$$F(1) = 1 * (7 - (60 \bmod 7))$$

$$F(1) = 1 * 3 = 3$$

ขั้น 1 $h_1(60) = (\text{hash}(x) + f(i)) \% \text{TableSize}; f(1) = 3$

$$h_1(60) = (0 + 3) \div 10 = 3; \leftarrow \text{ชนให้ } i+1$$

$$F(2) = 2 * \text{hash}_2(60)$$

$$F(2) = 2 * (7 - (60 \bmod 7))$$

$$F(2) = 2 * 3 = 6$$

ขั้น 2 $h_2(60) = (\text{hash}(x) + f(i)) \% \text{TableSize}; F(2) = 6$

$$h_2(60) = (0 + 6) \div 10 = 6; \leftarrow \text{ชนให้ } i+1$$

$$F(3) = 3 * \text{hash}_2(60)$$

$$F(3) = 3 * (7 - (60 \bmod 7))$$

$$F(3) = 3 * 3 = 9$$

ขั้น 3 $h_3(60) = (\text{hash}(x) + f(i)) \% \text{TableSize}; F(3) = 9$

$$h_3(60) = (0 + 9) \div 10 = 9; \leftarrow \text{ชนให้ } i+1$$

$$F(4) = 4 * \text{hash}_2(60)$$

$$F(4) = 4 * (7 - (60 \bmod 7))$$

$$F(4) = 4 * 3 = 12$$

ขั้น 4 $h_4(60) = (\text{hash}(x) + f(i)) \% \text{TableSize}; F(4) = 12$

$$h_4(60) = (0 + 12) \div 10 = 2; \leftarrow \text{ไม่ชนแล้ว}$$

60 จะมีการชนกันและแก้การชนกัน 4 ครั้ง และจะลงตำแหน่งที่ 2 ใน HashTable

	After 69
0	69
1	
2	
3	58
4	
5	
6	49
7	
8	18
9	89

	After 69
0	69
1	
2	60
3	58
4	
5	
6	49
7	
8	18
9	89

Rehashing

เมื่อจำนวนข้อมูลที่ใส่ใน hashTable มากขึ้น จำนวนครั้งที่ตรวจสอบหาช่องว่างใหม่จะมากขึ้นตามไปด้วย เวลาที่ใช้จะนานขึ้น และ ทำให้ใส่ข้อมูลยากขึ้น **ใส่ไม่ได้** เช่น 23

วิธีแก้ : จะสร้าง HashTable ใหม่ที่ใหญ่กว่าเดิม 2 เท่า เรียกว่า **Rehashing**

ยกตัวอย่าง : ใส่ข้อมูล 13, 15, 24, 6 ใน HashTable 7 ช่อง
ใช้ Hash ฟังก์ชันคือ $hash(x) = x \bmod TableSize$
ใช้ แบบ Linear probing กัน

0	6
1	15
2	
3	24
4	
5	
6	13

จำนวนช่องใน Hash Table มี 7 ช่อง ใช้ไป 4 ช่อง
จำนวนช่องใน Hash Table มี 100 คิดเป็น $4 \times 100 / 7 = 57.14\%$

ถ้าใส่ 23 ด้วย จำนวนช่องใน Hash Table มี 7 ช่อง ใช้ไป 5 ช่อง
จำนวนช่องใน Hash Table มี 100 คิดเป็น $5 \times 100 / 7 = 71.42\%$

ถ้าข้อมูลมีมากกว่า 70% ให้ Rehashing

↑
แล้วตัดใจยกกำหนด

* สมมติ TableSize มี 7 ใหญ่กว่า 2 เท่า = 14 ,
จะไม่ใช้ 14 จะใช้เลขจำนวนเฉพาะที่น้อยที่สุด
แต่มากกว่า หรือ เท่ากับ 2 เท่าของ HashTable เดิม เช่น

2	3	5	7	11	13	17	19	23
29	31	37	41	43	47	53	59	61
67	71	73	79	83	89	97		

7 11 13 **14** 17 19 23
↑
เอาอันนี้

และ อ่านข้อมูลจาก HashTable ตามลำดับคือ 6, 15, 23, 24, 13
และ ใช้ $H_{new}(x) = 6 \bmod 23$ * ห้ามเอาลำดับข้อมูลเดิม

main program

enum ย่อมาจาก คำว่า enumeration หมายถึง ใช้ตัว
struct ย่อมาจาก คำว่า structure แปลว่า โครงสร้าง

```
int main() {  
    1 HashTable<int> a(-999, 10)  
    2 int x  
    3 a.insert(32);  
    4 a.insert(49);  
    5 a.remove(32);  
    6 x = a.find(49);  
    7 if (x == -999) cout << "Not Found" << endl;  
    8 else cout << "Found" << endl;  
}
```

บรรทัดที่ 1

* ในข้อสอบเขียนแค่ในกรอบ จัณตอนที่ 1 และ 2 เท่านั้น

จันตอนที่ 1

ระบบปฏิบัติการคอมพิวเตอร์ จะสร้างตัวแปรออบเจกต์ a ขึ้นในหน่วยความจำตามโครงสร้าง คลาส HashTable ชนิดข้อมูล Int จะถูกส่งไปที่ Class template ชื่อ HashedObj ตรงไหนเขียนคำว่า HashedObj ในโครงสร้าง คลาส HashTable ก็จะถูกเปลี่ยนเป็นคำว่า Int สำหรับตัวแปร ออบเจกต์ a และส่งค่าพารามิเตอร์เข้าไป 2 ค่า คือ -999 กับ 10

a

public

private

array =
currentSize =
ITEM_NOT_FOUND =

```
void HashTable<HashedObj>::makeEmpty()
{
    currentSize = 0;
    for( int i = 0; i < array.size(); i++ )
        array[ i ].info = EMPTY;
}
```

ฟังก์ชัน nextPrime (size = 10)

คือ ฟังก์ชัน หาจำนวนเฉพาะ ที่น้อยที่สุดแต่ $\geq$ size
 nextPrime (10) = 11
 nextPrime (11) = 11 เพราะเป็นเลขจำนวนเฉพาะ ที่ = size

ค่าตัวแปรตอนนี้

a

public

private

array = 1111
 currentSize = 0
 ITEM_NOT_FOUND = -999

จันตอนที่ 2

ระบบปฏิบัติการจะทำงาน Constructure function ของ HashTable

```
template <class HashedObj>
HashTable<HashedObj>::
    HashTable( const HashedObj & notFound, int size )
        :ITEM_NOT_FOUND( notFound ), array( nextPrime( size ) )
    {
        makeEmpty();
    }
```

ระบบปฏิบัติการจะไปจองพื้นที่ในหน่วยความจำ ที่ติดกัน จำนวน 11 ช่อง (ที่ได้จากฟังก์ชัน nextPrime) แต่ละช่องประกอบตัวโครงสร้าง 2 ส่วน คือ element กับ info นึกว่า Empty ไปกินไว้ ส่วนที่ชื่อว่า info ของแต่ละช่อง

1111	element	info
0	E	
1	E	
2	E	
3	E	
4	E	
5	E	
6	E	
7	E	
8	E	
9	E	
10	E	

บรรทัดที่ 3

a.insert(b2);

```
void HashTable<HashedObj>::insert( const HashedObj & x ) {
    int currentPos = findPos( x );
    if( isActive( currentPos ) )
        array[ currentPos ] = HashEntry( x, ACTIVE );
    if ( ++currentSize > array.size() / 2 )
        rehash();
    bool HashTable<HashedObj>::isActive( int currentPos ) const
    {
        return array[ currentPos ].info == ACTIVE;
    }
```

```
HashEntry( const HashedObj & e = HashedObj(),
    EntryType i = EMPTY ) : element( e ), info(i) { }
```

array[currentPos] = hashEntry(x, ACTIVE); ฟังก์ชัน HashEntry() คือ รับค่าข้อมูลไปเก็บไว้ใน Array[currentPos] นั่นเอง เราสามารถเขียนประโยคคำสั่ง (statement) เพื่อใช้แทนฟังก์ชัน HashEntry ได้ดังนี้ array[currentPos].element = x; array[currentPos].info = ?;

```
/* Method that performs quadratic probing resolution. f(i) = i^2
template <class HashedObj>
int HashTable<HashedObj>::findPos(const HashedObj & x) const
{
    int collisionNum = 0; // ก็คือ ตัวแปร i ในสมการ นั่นเอง
    int currentPos = hash( x, array.size() );
    currentPos = 5;
    while ( array[ currentPos ].info != EMPTY
        array[ currentPos ].element != x ) {
        currentPos += 2 * ++collisionNum - 1;
        if( currentPos >= array.size() )
            currentPos -= array.size();
    }
    return currentPos;
}
```

```
template <class HashedObj>
int hash( const HashObj x, int tableSize ) {
    return x % tableSize;
}
```

บรรทัดที่ 4 a.insert(49);

```
void HashTable<HashedObj>::insert( const HashedObj & x ) {
    int currentPos = findPos( x );
    if( isActive( currentPos ) ) เป็นเท็จ ไม่ทำ return;
    array[ currentPos ] = HashEntry( x, ACTIVE );
    if ( ++currentSize > array.size() / 2 )
        rehash(); ไม่ทำ
```

```
/* Method that performs quadratic probing resolution. f(i) = i^2
template <class HashedObj>
int HashTable<HashedObj>::findPos(const HashedObj & x ) const
{
    int collisionNum = 0; //ก็คือ ตัวแปร i ในสมการ นั้นเอง
    int currentPos = hash( x, array.size() );
    currentPos = 5;
    while ( array[ currentPos ].info != EMPTY &&
            array[ currentPos ].element != x ) {
        currentPos += 2 * ++collisionNum - 1;
        6 = 5 + 2 * 1 - 1
        if( currentPos >= array.size() )
            currentPos -= array.size();
    }
```

```
    while ( array[ currentPos ].info != EMPTY &&
            array[ currentPos ].element != x ) {
        currentPos += 2 * ++collisionNum - 1;

        if( currentPos >= array.size() )
            currentPos -= array.size();
    }
    return currentPos; 6
}
```

ตอนนี้ได้ดังนี้

1111	element	info
0		E
1		E
2		E
3		E
4		E
5	82	E
6	49	E
7		E
8		E
9		E
10		E

* และต้องมี 6 ตัวขึ้นไปถึงจะ Rehashing

บรรทัดที่ 5 a.remove(82);

```
template <class HashedObj>
void HashTable<HashedObj>::remove( const HashedObj & x )
{
    int currentPos = findPos( x );
    if( isActive( currentPos ) )
        array[ currentPos ].info = DELETED;
        currentSize--;
}
```

```
bool HashTable<HashedObj>::isActive( int currentPos ) const
{
    return array[ currentPos ].info == ACTIVE;
}
```

```
/* Method that performs quadratic probing resolution. f(i) = i^2
template <class HashedObj>
int HashTable<HashedObj>::findPos(const HashedObj & x ) const
{
    int collisionNum = 0; //ก็คือ ตัวแปร i ในสมการ นั้นเอง
    int currentPos = hash( x, array.size() );
    currentPos = 5;
    while ( array[ currentPos ].info != EMPTY &&
            array[ currentPos ].element != x ) {
        currentPos += 2 * ++collisionNum - 1;

        if( currentPos >= array.size() )
            currentPos -= array.size();
    }
    return currentPos; 5
}
```

ตอนนี้ได้ดังนี้

1111	element	info
0		E
1		E
2		E
3	27	E
4		E
5	82	D
6	49	E
7		E
8		E
9	71	E
10		E

บรรทัดที่ 6 $x = a.find(49);$

```
const HashedObj & HashTable<HashedObj>::
find( const HashedObj & x = ) const {
    int currentPos = findPos( x = );
    if( isActive( currentPos ) )
        return array[ currentPos ].element;
    else return ITEM_NOT_FOUND;
}

/* Method that performs quadratic probing resolution.  $f(i) = i^2$ 
template <class HashedObj>
int HashTable<HashedObj>::findPos(const HashedObj & x ) const
{
    int collisionNum = 0; //ก็คือ ตัวแปร i ในสมการ นั่นเอง
    int currentPos = hash( x, array.size( ) );
    currentPos = 5;
    รอบ 1 while ( array[ currentPos ].info != EMPTY
        array[ currentPos ].element != x ) {
        currentPos += 2 * ++collisionNum - 1;
        // 6 = 5 + 2 * 1 - 1
        if( currentPos >= array.size( ) )
            currentPos -= array.size( );
    }
    รอบ 2 while ( array[ currentPos ].info != ACTIVE
        array[ currentPos ].element != x ) {
        currentPos += 2 * ++collisionNum - 1;
        if( currentPos >= array.size( ) )
            currentPos -= array.size( );
    }
    return currentPos;
}
```

bool HashTable<HashedObj>::isActive(int currentPos) const
{
 return array[currentPos].info == ACTIVE;
}

ถ้าค้นหา 49 จะได้ -999 เพราะสถานะของ 49 เป็น Delete แล้วไม่ได้เป็น ACTIVE

Priority Queue หรือ Heap

จะมี 2 Operation

Insert จะเหมือน enqueue ของ Queue

deleteMin จะเป็นการหาค่าที่น้อยที่สุดและค่อยลบใน Queue
จะเหมือน dequeue ของ Queue

จะใช้งานทาง Computer

- ตัวเครื่องปรีน
- OS ของคอมพิวเตอร์

Binary Heap หรือเรียกว่า Heaps มี 2 คุณสมบัติ

- 1.) คุณสมบัติเชิงโครงสร้าง
- 2.) คุณสมบัติเชิงอันดับ

คุณสมบัติเชิงโครงสร้าง

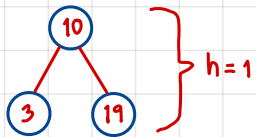
คือ Binary Heaps ที่สมบูรณ์ คือ
จะเติมหมดทุกชั้น ยกเว้นชั้นล่างสุดไม่ต้องเติมก็ได้

ใส่ จัอมล จะใส่จากล่างไล่จากซ้ายไปขวา

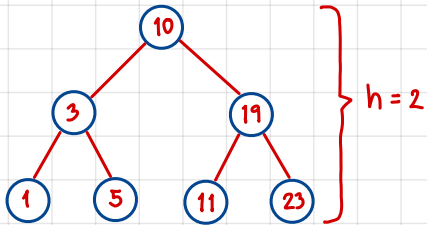
ลบ จัอมล จะไล่จากล่าง จากขวาไปซ้าย

ถ้า heap มีความสูง = h จะมีจำนวน Node อยู่ระหว่าง 2^h และ $2^{h+1}-1$

ตัวอย่าง :

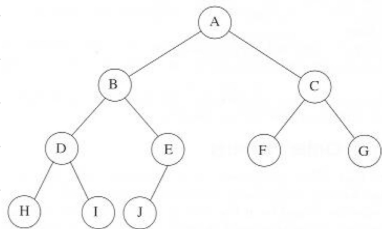


จำนวนโหนดอย่างน้อย $2^h = 2^1 = 2$ โหนด
จำนวนโหนดมากที่สุดเท่าที่ $2^{h+1}-1 = 2^{1+1}-1 = 3$ โหนด



จำนวนโหนดอย่างน้อย $2^h = 2^2 = 4$ โหนด
จำนวนโหนดมากที่สุดเท่าที่ $2^{h+1}-1 = 2^{2+1}-1 = 7$ โหนด

Complete binary tree สามารถทำเป็น Array ได้
จะไม่เริ่มที่ตำแหน่งที่ 0 จะเริ่มจากบนลงล่าง จากซ้ายไปขวา
เพราะมันจะใช้สูตรไม่ได้ root ต้องอยู่ตำแหน่งที่ 0 เท่านั้น



ถ้าพิจารณา D index D คือ i
left child = $2^i = 2^4 = 8$
Right child = $2^i+1 = 2^4+1 = 9$
Parent = $i/2 = 4/2 = 2$
ผลลัพธ์มีเลขปัดทิ้ง

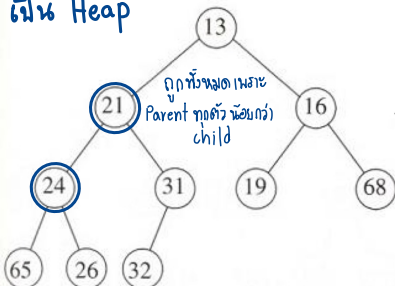
X	A	B	C	D	E	F	G	H	I	J				
0	1	2	3	4	5	6	7	8	9	10	11	12	13	

* ออกสอบ

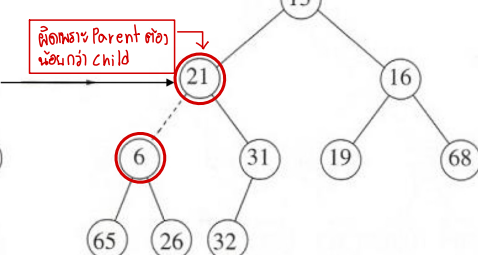
Complete Binary Tree มีความสูง = 15
จำนวนโหนดอย่างน้อย $2^{15} = 32,768$ โหนด
จำนวนโหนดมากที่สุด $2^{16}-1 = 65,535$ โหนด
ถ้าโจทย์ไม่มี Complete ให้สูตรนี้ไม่ได้

คุณสมบัติเชิงอันดับ สำหรับ Node x ใดๆ ค่าใน Parent จะต้องน้อยกว่า x (ยกเว้น root)

เป็น Heap



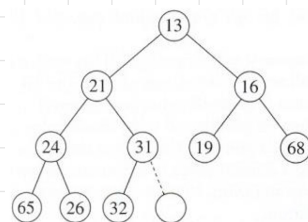
ไม่เป็น Heap



main program

```
int main() {
    1 BinaryHeap<int> a(12);
    2 int x;
    3 a.insert(14);
    4 a.deleteMin(x);
}
```

1111	[0]	ไม่มีข้อมูล
1	13	
2	21	
3	16	
4	24	
5	31	
6	19	
7	68	
8	65	
9	26	
10	32	
11		
12		



ออกแบบจาก a มีสมาชิก 10 ตัว

บรรทัดที่ 1 * ในข้อสอบเขียนแค่ในกรอบ ขั้นตอนที่ 1 และ 2 เท่านั้น

ขั้นตอนที่ 1

ระบบปฏิบัติการคอมพิวเตอร์ จะสร้างตัวแปรอัตโนมัติ a ขึ้นในหน่วยความจำตามโครงสร้าง คลาส BinaryHeap ชนิดข้อมูล Int จะถูกส่งไปที่ Class template ชื่อ Comparable ตรงไหนเขียนคำว่า Comparable ในโครงสร้าง คลาส BinaryHeap ก็จะถูกเปลี่ยนเป็นคำว่า Int สำหรับตัวแปรอัตโนมัติ a และส่งค่าพารามิเตอร์เข้าไป 1 ค่า คือ 12

a

public

.....

.....

private

currentSize =

array =

ขั้นตอนที่ 2

ระบบปฏิบัติการจะไปทำงาน Constructure function ของ BinaryHeap

```
template <class Comparable>
BinaryHeap<Comparable>::BinaryHeap( int capacity )
: array( capacity + 1 ), currentSize( 0 ) { }
```

ตัวแปรพารามิเตอร์ ชื่อ capacity = 12

ระบบปฏิบัติการไปจองพื้นที่ในหน่วยความจำ ติดกัน = capacity + 1 = 13

แต่ละข้อมูลขนาดเท่ากับชนิดข้อมูล int เก็บ address ของหน่วยความจำตำแหน่งที่ 0 ไว้ใน

Data member ชื่อ array

1111	[0]	ไม่มีข้อมูล
1		
2		
3		
4		
5		
6		
7		
8		
9		
10		
11		
12		

ตอนนี้ได้ดังนี้

a

public

.....

.....

private

currentSize = 0

array = 1111

ขั้นตอนวิธี insert

- สร้าง Hole (วงกลม) ที่ตำแหน่งที่ต้องการไปของ Array
- ขณะที่ใส่ x ใน Hole จะให้ตรวจสอบ Heap Order
 - ถ้าเช็คแล้ว x มีค่าน้อยกว่า Parent ให้สลับ (Swapping) ของ Hole ไปไว้ตำแหน่ง Parent และตำแหน่ง Parent ย้ายมาตำแหน่งที่ child และวนกลับไปตรวจสอบ Heap Order อีกครั้ง
 - ถ้าเช็คแล้วค่า x มีค่ามากกว่า Parent ให้ใส่ค่า x ใน Hole ได้เลย

บรรทัดที่ 3 a.insert(14);

```
bool BinaryHeap<Comparable>::isFull( ) const {
    return currentSize == array.size( ) - 1;
}
```

10 เท็จ 13-1

```
void BinaryHeap<Comparable>::insert( const Comparable&x14 )
{
    if( isFull( ) )
        throw Overflow( );   ยังไม่ทำเพราะ if isfull เท็จ

    // Percolate up
    int hole = ++currentSize;
    for( ; hole > 1 && x < array[ hole / 2 ]; hole /= 2 )
        array[ hole ] = array[ hole / 2 ];
    array[ hole ] = x;
}
```

hole = 5

การทำงานของฟังก์ชัน Insert

* ออกสอ

hole = 11

for loop รอบที่ 1 Hole = 11 > 1 จริง เปรียบเทียบ 14 < 31 จริง
 for loop รอบที่ 2 Hole = 5 > 1 จริง เปรียบเทียบ 14 < 21 จริง
 for loop รอบที่ 3 Hole = 2 > 1 จริง เปรียบเทียบ 14 < 13 เท็จ
 ออกจาก for loop ตัวแปร hole มีค่าเท่ากับ 2
 for loop ทำงาน 3 รอบ ประโยคคำสั่งที่อยู่ใน for loop ทำงาน 2 รอบ

1111 [0] ไม่มีข้อมูล

1	13
2	14
3	16
4	29
5	21
6	19
7	68
8	65
9	26
10	32
11	31
12	

บรรทัดที่ 4 a.deleteMin(x);

```
void BinaryHeap<Comparable>::deleteMin( Comparable & minItem ) {
    if( isEmpty( ) ) throw Underflow( );
    minItem = array[ 1 ];
    array[ 1 ] = array[ currentSize-- ];   ลบค่าคือ 10
    percolateDown( 1 );
}
```

```
bool BinaryHeap<Comparable>::isEmpty( ) const {
    return currentSize == 0;
}
```

11 เท็จ 0

```
void BinaryHeap<Comparable>::percolateDown( int hole ) {
    int child;
    Comparable tmp = array[ hole ];
    for( ; hole * 2 <= currentSize; hole = child ) {
        child = hole * 2;
        if( child != currentSize && array[child + 1] < array[child] ) child++;
        if( array[ child ] < tmp ) array[ hole ] = array[ child ];
        else break;
    }
    array[ hole ] = tmp;
}
```

การทำงานของฟังก์ชัน deleteMin

* ออกสลับ

```
hole = 11    tmp 31
for loop รอบที่ 1 Hole = 1 , child = 2   เปรียบเทียบ 16 < 14   เท็จ
for loop รอบที่ 2 Hole = 2 , child = 4   เปรียบเทียบ 21 < 19   เท็จ ไม่ทำ child++;
for loop รอบที่ 3 Hole = 4 , child = 8   เปรียบเทียบ 26 < 65   จริง  ทำ child++;
for loop รอบที่ 4 Hole = 9 , child = 9
for loop ทำงาน 3 รอบ
```

1111	[0]	ไม่มีข้อมูล
1	14	
2	19	
3	16	
4	26	
5	21	
6	19	
7	68	
8	65	
9	31	
10	32	
11	31	
12		

Sorting

* เราจะเรียงจากน้อยไปหามากเท่านั้น *

- Algorithms ทั้งหมด สามารถสลับ สับเปลี่ยนกันได้ โดยไม่กระทบข้อมูล และ ผลลัพธ์
- การเรียงลำดับจะทำงานใน Array ทั้งหมด
- จะใช้เครื่องหมาย ">" และ "<" ในการเรียงลำดับข้อมูล
- จะเรียงลำดับในรูปแบบ Comparison-base sorting (การเปรียบเทียบ)

ตัวอย่างการเรียงลำดับ

Original	34	8	64	51	32	21	P. Moved
After p = 1	8	34	64	51	32	21	1
After p = 2	8	34	64	51	32	21	0
After p = 3	8	34	51	64	32	21	1
After p = 4	8	32	34	51	64	21	3
After p = 5	8	21	32	34	51	64	4

* ออกสลับ

อธิบายหลักการทำงานของ exchangeable ?

ตอบ สามารถเปลี่ยนชื่อของฟังก์ชันได้เลยเพราะไม่กระทบส่วนอื่นของ main program

main program

```
#include <iostream>;
using namespace std;
void insertionSort (vector<Comparable> &a);
int main() {
    vector<int> x(6);
    x[0] = 34;
    x[1] = 8;
    x[2] = 64;
    x[3] = 51;
    x[4] = 32;
    x[5] = 21;
    insertionSort(x);
} ต่อด้วย
```

```
template <class Comparable>
void insertionSort( vector<Comparable> &a )
{
    1 for( int p = 1; p < a.size(); p++ )
    {
        2 Comparable tmp = a[ p ];

        int j;
        3 for( j = p; j > 0 && tmp < a[ j - 1 ]; j-- )
        4 a[ j ] = a[ j - 1 ];
        5 a[ j ] = tmp;
    }
}
```

x=a 0 1 2 3 4 5
 34 8 64 51 32 21

For วง P รอบ 1 P = 1 1 < 6 จริง
 tmp = 8
 For วง j รอบ 1 j = 1 1 > 0 && 8 < 34 จริง
 a[1] = 34
 For วง j รอบ 2 j = 0 0 > 0 เท็จ
 a[0] = 8
 After p = 1 8 34 64 51 32 21 position move = 1

For วง P รอบ 2 P = 2 2 < 6 จริง
 tmp = 64
 For วง j รอบ 1 j = 2 2 > 0 && 64 < 34 เท็จ
 a[2] = 64
 After p = 2 8 34 64 51 32 21 position move = 0

For วง P รอบ 3 P = 3 3 < 6 จริง
 tmp = 51
 For วง j รอบ 1 j = 3 3 > 0 && 51 < 64 จริง
 a[3] = 64
 For วง j รอบ 2 j = 2 2 > 0 && 51 < 34 เท็จ
 a[2] = 51
 After p = 3 8 34 51 64 32 21 position move = 1

For วง P รอบ 4 P = 4 4 < 6 จริง
 tmp = 32
 For วง j รอบ 1 j = 4 4 > 0 && 32 < 64 จริง
 a[4] = 64
 For วง j รอบ 2 j = 3 3 > 0 && 32 < 51 จริง
 a[3] = 51
 For วง j รอบ 3 j = 2 2 > 0 && 32 < 34 จริง
 a[2] = 34
 For วง j รอบ 4 j = 1 1 > 0 && 32 < 8 เท็จ
 a[1] = 32
 After p = 3 8 32 34 51 64 21 position move = 3

For วง P รอบ 5 P = 5 5 < 6 จริง
 tmp = 21
 For วง j รอบ 1 j = 5 5 > 0 && 21 < 64 จริง
 a[5] = 64
 For วง j รอบ 2 j = 4 4 > 0 && 21 < 51 จริง
 a[4] = 51
 For วง j รอบ 3 j = 3 3 > 0 && 21 < 34 จริง
 a[3] = 34
 For วง j รอบ 4 j = 2 2 > 0 && 21 < 32 จริง
 a[2] = 32
 For วง j รอบ 4 j = 1 1 > 0 && 21 < 8 เท็จ
 a[1] = 21
 After p = 3 8 21 32 34 51 64 position move = 4

มีความเป็นไปได้ 3 กรณี

1. Best Case (กรณีที่ดีที่สุด) คือ ข้อมูลที่ Position move น้อยที่สุด เริ่มจาก น้อย - มาก
 2. Worst Case (กรณีที่ไม่ดีที่สุด) คือ ข้อมูลที่ Position move มาก ที่สุด เริ่มจาก มาก - น้อย
 3. Average Case (กรณีทั่วไป)
- คนจะคำนึงถึง Worst Case เป็นหลัก

Shellsort

ผู้สร้าง Algorithm นี้ชื่อว่า Donal Shell ในวิธีแรกที่เราจะ Insertionsort

จะใช้ Sequence เพื่อทำระยะห่างของตัวลง เรียกว่า Increment sequence มี 3 แบบ 1, 3, 5

จะเริ่มตัวแรกและนับไปอีก (ไม่นับตัวแรก) จนสุด แล้วเปรียบจากน้อยไปมาก แล้วขยับช่องแล้วทำจนครบ จะไม่ทำตัวที่เรียงแล้ว

แบบ 5

Original	81	94	11	96	12	35	17	95	28	58	41	75	15
After 5-sort	35	17	11	28	12	41	75	15	96	58	81	94	95

แบบ 3 (ใช้ตัวเลขที่เรียงแบบ 5 มาแล้ว)

Original	81	94	11	96	12	35	17	95	28	58	41	75	15
After 5-sort	35	17	11	28	12	41	75	15	96	58	81	94	95
After 3-sort	28	12	11	35	15	41	58	17	94	75	81	96	95

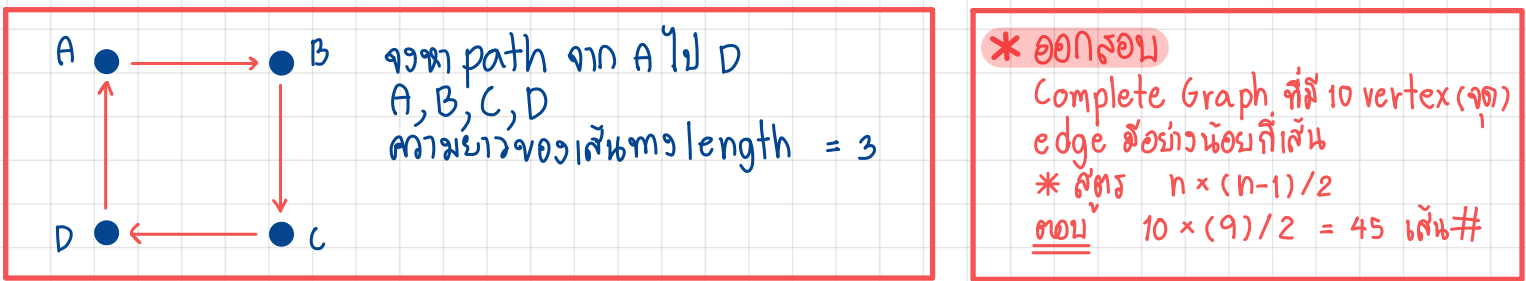
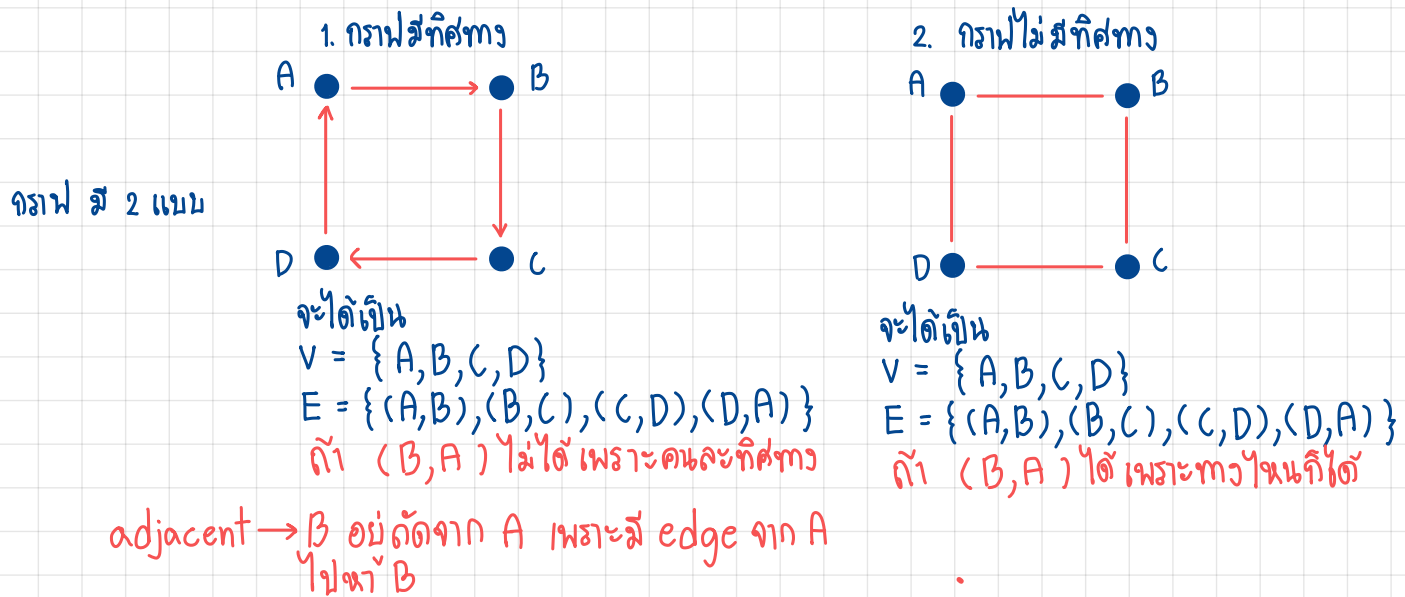
Graphs

กราฟ คือ ความสัมพันธ์ ระหว่าง จุด และ เส้น

$G = (V, E)$ ต้องเขียนเซตให้ได้ $V = \text{vertices (จุด)}$
 $E = \text{edges (เส้น)}$

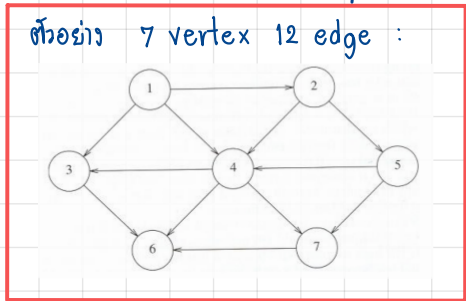
- ถ้าจุดเริ่มต้นและจุดสุดท้ายเป็นข้อเดียวกัน เรียกว่า loop
- simple path คือ path ใดๆ จุด ไม่ซ้ำกัน ยกเว้น จุด เริ่มต้นและจุดสุดท้ายเหมือนกันได้
- cycle คือจุดเริ่มต้นและจุดสุดท้ายเป็นข้อเดียวกัน
- acyclic คือจะไม่ cycle
- connected คือ มาต่อกันได้ ตลอดทั้งหมด สังเกตจาก Path (เส้นทาง)
- Strongly Connect คือ มาต่อกันได้ มี Path ทั้งหมด
- weakly Connect คือ ทำ path ของ Path ออก
- Complete graph คือ กราฟที่มี เส้นอยู่ระหว่าง จุดทั้งหมด



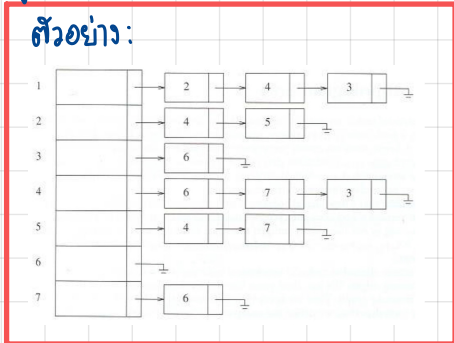


Representation Graph วิธีที่ 1 คอมพิวเตอร์เข้าใจกราฟ ?

- จะเริ่มต้นด้วย directed graph
- จะใช้ ตัวเลข แทน Vertex (จุด) และจะเริ่มต้นที่ 1



- Adjacency List (นิยมใช้)
 - ถ้ากราฟจำนวนเส้นน้อย เรียกว่า กราฟเบาบาง (sparse)
 - จะใช้เป็น Linkedlist แทน
- สูตร $O(|E| + |V|) = O(\text{ผลรวม } E \text{ และ } V)$



- Adjacency Matrix (ไม่นิยมใช้)
 ใช้ Array 2 มิติ แทน กราฟ แต่ไม่นิยมใช้
 เพราะสิ้นเปลืองหน่วยความจำ
 สูตร $\Theta(|V|^2) = \Theta(\text{ผลรวม vertex}^2)$

ตัวอย่าง Array 2 มิติ :

		1	2	3	4	5	6	7
1 จาก	1	0	1	1	1	0	0	0
	2	0	0	0	1	1	0	0
	3	0	0	0	0	0	1	0
	4	0	0	1	0	0	1	1
	5	0	0	0	1	0	0	1
	6	0	0	0	0	0	0	0
	7	0	0	0	0	0	1	0

จากตัวอย่าง 49 ช่อง มีช่องเป็น 1 อยู่ 12 ช่อง
 คิดเป็นเปอร์เซ็นต์ คือ $12 \times 100 / 49 = 24.48\%$
 ดังนั้น ใช้ช่องแค่ 24.48% เท่านั้นเหลืออีก
 75.52% ที่ไม่ได้ใช้งาน

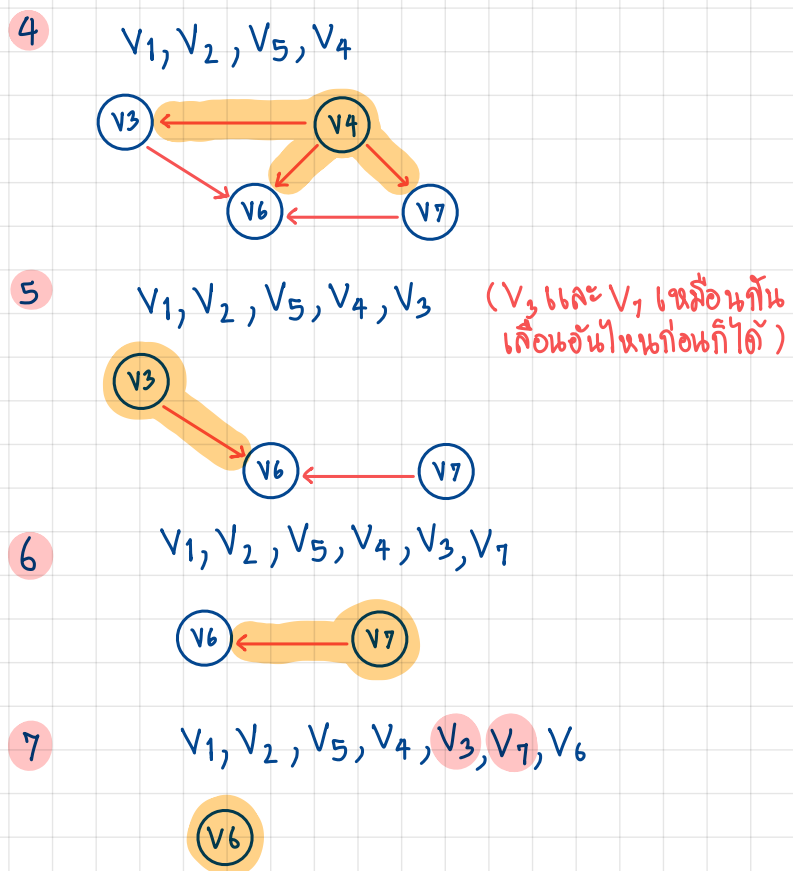
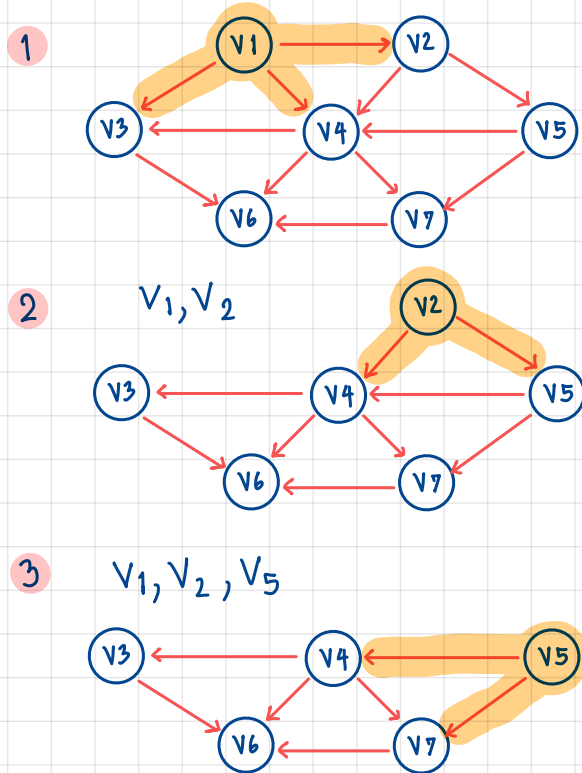
Topological Sort

คือ การเรียงลำดับของ Vertex (จุด) ที่อยู่ใน directed acyclic graph
(กราฟมีทิศทางไม่มีวงรอบ)

- ถ้ากราฟมีวงรอบจะเขียนกราฟ Topological sort ไม่ได้ (cycle)

- วิธีทำ
1. เริ่มด้วย Vertex ที่ไม่มี edge เข้า
 2. หา Vertex ที่ไม่มี edge เข้า
 3. เมื่อเจอแล้ว ลบเส้นและจุดที่เจอแล้วออก และหาต่อไปจนครบ

ตัวอย่าง : V_1



ตอบ $V_1, V_2, V_5, V_4, V_3, V_7, V_6$

Algorithm

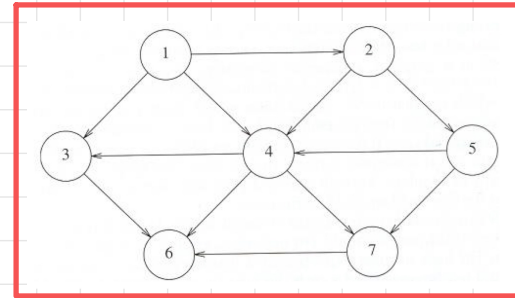
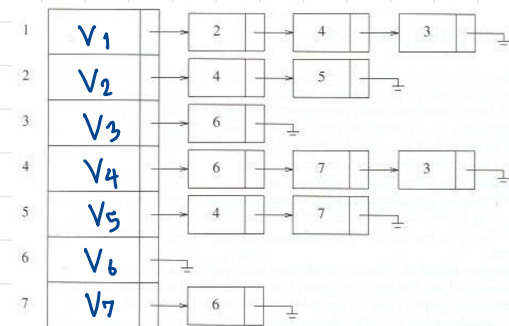
ทำซ้ำ

- หา Vertex ที่ไม่มี edge เข้า
- แสดง Node นั้นออกมา
- ลบ Node ที่พบออก

ทำจนกว่ากราฟจะว่าง

ขั้นตอนการทำตาราง Indegree

1. แปลงกราฟมาเป็น adjacency list ก่อน



2. ทำ Indegree array ของจำนวน edge ที่ชี้เข้า Vertex

vertex	เริ่มต้น	รอบ 1	รอบ 2	รอบ 3	รอบ 4	รอบ 5	รอบ 6	q
1	0							6
2	1							5
3	2							4
4	3							3
5	1							2
6	3							1
7	2							0

currentSize = 0
front = 0
back = -1

จำนวนเส้นชี้เข้า

พอเริ่มในรอบที่ 1 จะชี้ได้

```

void Graph::topsort() {
    Queue q(NUM_VERTICES);
    int counter = 0;
    Vertex v, w;

    q.makeEmpty();
    for each vertex v
        if (v.indegree == 0)
            q.enqueue(v);

    while (!q.isEmpty())
    {
        v = q.dequeue();
        v.topNum = ++counter; // assign next number

        for each w adjacent to v
            if (--w.indegree == 0)
                q.enqueue(w);
    }

    if (counter != NUM_VERTICES)
        throw CycleFound();
}
  
```

ส่วนนี้อธิบายค่อนข้างมาก
แนะนำให้ดูคลิปอาจารย์สอน
แล้วทำความเข้าใจ
(week 6 part 1 59:54)

เมื่อทำเสร็จแล้วจะได้ตัวนี้

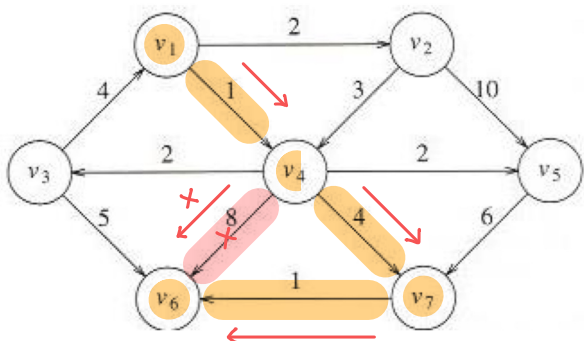
Vertex	Indegree Before Dequeue #						
	1	2	3	4	5	6	7
v_1	0	0	0	0	0	0	0
v_2	1	0	0	0	0	0	0
v_3	2	1	1	1	0	0	0
v_4	3	2	1	0	0	0	0
v_5	1	1	0	0	0	0	0
v_6	3	3	3	3	2	1	0
v_7	2	2	2	1	0	0	0
Enqueue	v_1	v_2	v_5	v_4	v_3, v_7		v_6
Dequeue	v_1	v_2	v_5	v_4	v_3	v_7	v_6

Dequeue คือคำตอบของ Topological sort

Shortest Path

คือ เส้นทางที่สั้นที่สุด

ยกตัวอย่าง : จาก v_1 ไป v_6



จะได้เป็น v_1, v_4, v_7, v_6

ถ้าหาแบบ unweighted Shortest คือแบบไม่สนใจน้ำหนัก ส่วนใจความสั้นของเส้นทาง (Path) จะได้เป็น v_1, v_4, v_6

ปัญหา ของ Shortest Path

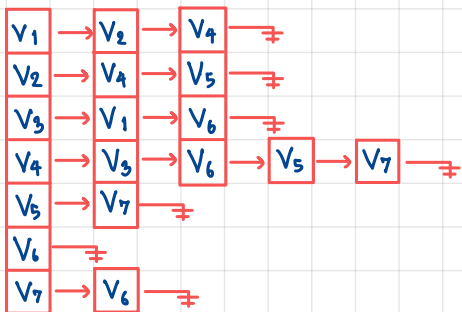
ไม่สามารถให้ edge (เส้นทาง) ติดกันได้

แต่เราจะเขียนแค่ในรูปแบบ Unweighted Shortest Path

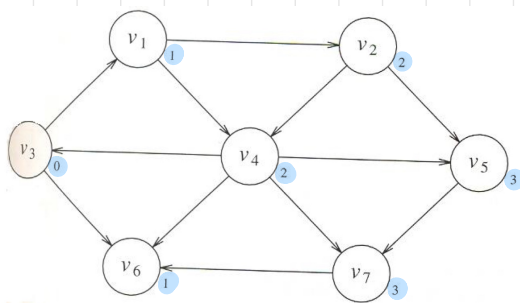
- โจทย์จะกำหนดเส้นทางมาให้จากจุดใดไปจุดใด
- กราฟมี cycle ก็ได้

ขั้นตอน วิธีในการทำ

1. แปลงกราฟมาเป็น adjacency list ก่อน



สมมติ กำหนดเริ่มที่ v_3 และหาเส้นทางไปทุกๆ v_n เพื่อได้ระยะทางไว้ใส่ไว้ที่ v_n



2. สร้าง อาร์เรย์สำหรับใช้ประมวลผล Shortest Path

	Known	Dist (Dv)	Path (Pv)	
1	F	999	0	6
2	F	999	0	5
3	F	999	0	4
4	F	999	0	3
5	F	999	0	2
6	F	999	0	1
7	F	999	0	0

* Dv คือ ระยะทาง

* Pv คือ เส้นทาง

3. ไล่ Code function Unweighted

```

/*Pseudocode for unweighted shortest-path
algorithm USE this Code*/
void Graph::unweighted( Vertex s ) 3 คือจุดเริ่มต้น
{
    Queue q (NUM_VERTICES); 7
    Vertex v, w;
    q.enqueue( s = 3 );
    s.dist = 0;
    while ( !q.isEmpty() )
    {
        v = q.dequeue ( );
        v.known = true;

        for each w adjacent to v
            if (w.dist == INFINITY 999)
            {
                w.dist = v.dist + 1;
                w.path = v;
                q.enqueue (w);
            }
    }
}
    
```

* วัตถุประสงค์นี้อาศัยความเข้าใจในการทำกรบ้าน *

* Assignment 4 ถ้าทำกรบ้านมาจะเข้าใจ *

ผลลัพธ์ได้ดังนี้

(ควรวทำ Assignment 4 ด้วย)

และดูคลิปประกอบ week 6 part 2 34:50

v	Initial State			v_3 Dequeued			v_1 Dequeued			v_6 Dequeued		
	known	d_v	p_v	known	d_v	p_v	known	d_v	p_v	known	d_v	p_v
v_1	F	∞	0	F	1	v_3	T	1	v_3	T	1	v_3
v_2	F	∞	0	F	∞	0	F	2	v_1	F	2	v_1
v_3	F	0	0	T	0	0	T	0	0	T	0	0
v_4	F	∞	0	F	∞	0	F	2	v_1	F	2	v_1
v_5	F	∞	0	F	∞	0	F	∞	0	F	∞	0
v_6	F	∞	0	F	1	v_3	F	1	v_3	T	1	v_3
v_7	F	∞	0	F	∞	0	F	∞	0	F	∞	0
Q:	v_3			v_1, v_6			v_6, v_2, v_4			v_2, v_4		

v	v_2 Dequeued			v_4 Dequeued			v_5 Dequeued			v_7 Dequeued		
	known	d_v	p_v	known	d_v	p_v	known	d_v	p_v	known	d_v	p_v
v_1	T	1	v_3	T	1	v_3	T	1	v_3	T	1	v_3
v_2	T	2	v_1	T	2	v_1	T	2	v_1	T	2	v_1
v_3	T	0	0	T	0	0	T	0	0	T	0	0
v_4	F	2	v_1	T	2	v_1	T	2	v_1	T	2	v_1
v_5	F	3	v_2	F	3	v_2	T	3	v_2	T	3	v_2
v_6	T	1	v_3	T	1	v_3	T	1	v_3	T	1	v_3
v_7	F	∞	0	F	3	v_4	F	3	v_4	T	3	v_4
Q:	v_4, v_5			v_5, v_7			v_7			empty		

แบบข้อสอบ

75 คะแนนเต็ม

ข้อที่ 1. Hashing (15 คะแนน)
- TableSize มากกว่า 12

2. Heap
- คล้าย Assignment 3
- มีข้อ Insert และ deleteMin เล็กน้อย
- มีเงื่อนไขของการทำงาน
- Hole มีค่าเท่าไร
- Child มีค่าเท่าไร
- เปรียบเทียบ เปรียบเทียบ กับเลขโดด

3. InsertionSort
- จงเขียนผลลัพธ์ After P = ?
- ค่าของตัวแปร tmp
- ค่าทั้งหมดของ Position Move

4. Shellsort
- จะให้เลข increment sequence เช่น 1,3,5 หรือ 2,4,6
- จงเขียนผลการทำงานหลังจาก After Sort 1 หรือ 2

5. Topological Sort
- จงเขียนค่า Indegree Array

6. Shortest Path
- จงเขียนค่าในตารางออกมา

7. คำถามเชิงอธิบาย
- จงเขียนประโยคคำสั่งเพื่อใช้แทน Function _____
- คำนวณ edge ใน Complete Graph
- เขียนขั้นตอนวิธี